

Lección 18: Environment & Secrets

Variables de Entorno y Secretos Seguros

Agenda

- ¿Por qué Variables de Entorno?
 - Tipos de Configuración
 - Validación con Zod
 - Secretos en Desarrollo vs Producción
 - Mejores Prácticas
 - Errores Comunes
-

El Problema

❌ Hardcoded Secrets:

```
// NEVER do this!  
const API_KEY = 'sk_live_abc123xyz'  
const DATABASE_URL = 'postgres://user:pass@prod.db'  
const STRIPE_SECRET = 'rk_live_...'  
  
// Pushed to Git → Exposed to world  
// In source code → Can't change without deploy  
// Same for all environments → Testing uses production!
```

Consecuencias:

- Credenciales expuestas en GitHub
- Producción comprometida
- Imposible rotar secretos

- Tests afectan datos reales
-

¿Por qué Variables de Entorno?

✓ Beneficios:

Separación de Responsabilidades:

- Código: Lógica de la aplicación
- Configuración: Settings específicos del entorno
- Secretos: Credenciales sensibles

Seguridad:

- No en control de código fuente
- Diferentes por entorno
- Fácil rotación
- Auditable

Flexibilidad:

- Cambio sin redeploy
 - Feature flags
 - Configuraciones A/B testing
-

Tipos de Configuración

1. Configuración Pública (puede estar en código):

```
const PUBLIC_CONFIG = {  
  APP_NAME: 'Shopping Cart',  
  VERSION: '2.1.0',  
  API_TIMEOUT: 5000,  
}
```

2. Específico del Entorno (variables de entorno):

```
const ENV_CONFIG = {  
  API_URL: process.env.VITE_API_URL,  
  ENVIRONMENT: process.env.NODE_ENV,  
}
```

3. Secretos (variables de entorno, nunca en código):

```
const SECRETS = {  
  DATABASE_URL: process.env.DATABASE_URL,  
  API_KEY: process.env.API_KEY,  
  JWT_SECRET: process.env.JWT_SECRET,  
}
```

¿Qué es Zod?

Zod: Librería de validación de schemas con TypeScript-first

¿Para qué sirve?

Valida datos en runtime + genera tipos en compile-time

Sin Zod:

=====

```
process.env.API_KEY  
→ string | undefined  
→ No hay garantías  
→ Crashes en producción
```

Con Zod:

=====

```
✓ Valida que existe  
✓ Valida formato correcto  
✓ TypeScript infiere tipo  
✓ Falla al inicio (fail-fast)
```

Ventajas:

- **✓ Type-safe:** TypeScript conoce los tipos automáticamente
- **✓ Fail-fast:** Detecta errores al iniciar la app
- **✓ Mensajes claros:** Dice exactamente qué está mal
- **✓ Transformaciones:** Convierte strings a números, URLs, etc.

Validación con Zod

Sin validación:

```
// Runtime crashes si falta variable  
const apiKey = process.env.API_KEY  
makeRequest(apiKey) // TypeError if undefined
```

Con validación Zod:

```
import { z } from 'zod'

const EnvSchema = z.object({
  API_KEY: z.string().min(1),
  DATABASE_URL: z.string().url(),
  PORT: z.string().transform(Number).pipe(z.number().positive()),
  NODE_ENV: z.enum(['development', 'production', 'test']),
})

// Valida al inicio
const env = EnvSchema.parse(process.env)

// TypeScript knows types!
env.PORT // number
env.NODE_ENV // 'development' | 'production' | 'test'
```

Validación: Beneficios

Falla temprana:

- ✗ Sin validación: App crashes en producción
- ✓ Con validación: App no inicia si config está mal

Mejor experiencia de desarrollo:

- Mensajes de error claros
- Seguridad de tipos
- Autocompletado en IDE
- Documentación vía schema

Ejemplo de error:

Error: Configuración de entorno inválida

Variables requeridas faltantes:

- DATABASE_URL

Variables inválidas:

- PORT: Se esperaba número, se recibió "abc"

Corrige estos errores antes de iniciar la aplicación.

Archivo .env

Desarrollo local:

```
# .env (not committed to git)
VITE_API_URL=http://localhost:3000
DATABASE_URL=postgres://localhost/dev_db
API_KEY=test_key_abc123
JWT_SECRET=local_dev_secret
```

.env.example (commiteado):

```
# .env.example (template for team)
VITE_API_URL=
DATABASE_URL=
API_KEY=
JWT_SECRET=
```

.gitignore:

```
.env
.env.local
.env.*.local
```

Desarrollo vs Producción

Desarrollo:

```
# .env.development
VITE_API_URL=http://localhost:3000
DATABASE_URL=postgres://localhost/dev_db
API_KEY=test_key_abc123 # Credenciales de prueba OK
SENTRY_DSN= # Opcional en desarrollo
```

Producción:

```
# Configurado en plataforma de hosting (Vercel, AWS, etc)
VITE_API_URL=https://api.production.com
DATABASE_URL=postgres://prod-server/prod_db
API_KEY=live_key_xyz789 # Credenciales reales
SENTRY_DSN=https://... # Requerido
```

Rotación de Secretos

Proceso:

1. Generar nuevo secreto (el viejo aún funciona)
2. Agregar nuevo secreto al entorno
3. Desplegar aplicación con ambos secretos
4. Verificar que el nuevo secreto funciona
5. Remover secreto viejo
6. Desplegar versión final

¡Rotación sin tiempo de inactividad!

Ejemplo:

```
// Soportar ambas API keys (vieja y nueva) durante rotación
const API_KEYS = [
  process.env.API_KEY_NEW, // Intentar nueva primero
  process.env.API_KEY_OLD, // Fallback a vieja
].filter(Boolean)

function isValidApiKey(key: string) {
  return API_KEYS.includes(key)
}
```

Mejores Prácticas

1. Nunca commitear secretos:

```
# Agregar a .gitignore inmediatamente
.env
.env.local
*.pem
*.key
```

2. Usar secretos diferentes por entorno:

```
Desarrollo: test_key_xxx
Staging: staging_key_yyy
Producción: live_key_zzz
```

3. Validar temprano:

```
// Al inicio de la app, no en runtime
EnvSchema.parse(process.env)
```

Mejores Prácticas (cont.)

4. Principio de Menor Privilegio:

```
// Dar los permisos mínimos necesarios
const READ_ONLY_DB = process.env.DATABASE_READ_URL
const WRITE_DB = process.env.DATABASE_WRITE_URL

// Usar solo lectura para queries
// Usar escritura solo cuando sea necesario
```

5. Auditar acceso a secretos:

```
// Loguear cuando se acceden secretos (¡no los valores!)
console.log('Accediendo a base de datos con credenciales')
// NO: console.log('DB URL:', dbUrl)
```

Errores Comunes

❌ Loguear secretos:

```
console.log('Config:', process.env) // ¡NUNCA!  
console.error('Failed with key:', apiKey) // ¡NUNCA!
```

❌ Secretos en mensajes de error:

```
throw new Error(`API key ${apiKey} is invalid`) // ¡NUNCA!  
// Correcto:  
throw new Error('API key is invalid')
```

❌ Secretos en URLs:

```
fetch(`/api/data?secret=${apiKey}`) // ¡NUNCA!  
// Correcto:  
fetch('/api/data', {  
  headers: { Authorization: `Bearer ${apiKey}` },  
})
```

Ejercicio 1: Environment Variables con Vite

Prompt:

Actúa como un developer externalizando configuración con environment variables

CONTEXT0: Hardcoded config = imposible cambiar sin redeploy. Environment variables (variables de entorno) = configuración externa al código. Vite prefix: VITE exponer vars al browser (seguridad: solo vars con VITE_ son públicas). Imponer objeto de Vite con env vars. .env file: archivo **local** con vars (NO commitar secrets). Fallback values: default si env var no existe (|| operator). 12-F metodología que requiere config en env (no en código). Beneficio: mismo código diferentes configs por entorno (dev/staging/prod).

TAREA: Reemplaza magic number MAX_CART_ITEMS con environment variable.

CREAR .ENV FILE:

- Archivo: .env (en root del proyecto)
- Variable: VITE_MAX_CART_ITEMS=99
- IMPORTANTE: Agregar .env a .gitignore (NO commitar)

USAR EN CÓDIGO:

- Archivo: src/hooks/useQuantityValidation.ts (o donde valides quantity)
- Leer: import.meta.env.VITE_MAX_CART_ITEMS
- Convertir a número: Number(import.meta.env.VITE_MAX_CART_ITEMS)
- Fallback: || 99 (si var no está definida)

IMPLEMENTACIÓN:

```
const MAX_ITEMS = Number(import.meta.env.VITE_MAX_CART_ITEMS) || 99
```

```
export function useQuantityValidation() {  
  const validate = (quantity: number) => {  
    if (quantity > MAX_ITEMS) {  
      return `Maximum ${MAX_ITEMS} items allowed`  
    }  
    return null  
  }  
  return { validate }  
}
```

TESTING:

1. Crear .env con VITE_MAX_CART_ITEMS=99
2. Ejecutar app: pnpm run dev
3. Intentar agregar 100 items → debe rechazar
4. Cambiar .env a VITE_MAX_CART_ITEMS=50
5. Reiniciar dev server (CTRL+C, pnpm run dev)
6. Intentar agregar 60 items → debe rechazar (límite cambió)

VALIDACIÓN: Límite debe cambiar sin modificar código, solo .env

Aprende: Environment variables externalizan config

permitiendo cambios sin redeploy

Ejercicio 2: Validación de Environment con Zod

Prompt:

Actúa como un platform engineer implementando runtime validation de environ

CONTEXT0: Environment vars son strings (process.env retorna strings). Sin v

errores en runtime cuando app espera número pero recibe string/undefined. Z librería de schema validation + TypeScript inference. Runtime validation: v en startup (NO compile time). Fail-fast principle: detectar config inválida de arrancar app (previene comportamiento impredecible). z.transform: convie string → number. z.pipe: encadena validaciones. parse() vs safeParse(): par lanza error, safeParse retorna result object. Centralized config: un lugar todas las env vars (src/config/env.ts).

TAREA: Valida VITE_MAX_CART_ITEMS con Zod schema al inicio de la app.

INSTALACIÓN:

- Comando: `pnpm add zod`
- Librería: Runtime schema validation

CREAR CONFIG FILE:

- Archivo: `src/config/env.ts`
- Purpose: Centralized + validated environment configuration

SCHEMA IMPLEMENTATION:

```
import { z } from 'zod'

const envSchema = z.object({
  VITE_MAX_CART_ITEMS: z.string() // Env vars son siempre strings
    .transform(Number) // Convertir a número
    .pipe(z.number().min(1).max(999)) // Validar rango
})

export const env = envSchema.parse({
  VITE_MAX_CART_ITEMS: import.meta.env.VITE_MAX_CART_ITEMS
})

// TypeScript infiere tipo: env.VITE_MAX_CART_ITEMS es number (no string!)
```

INTEGRACIÓN EN APP:

- Archivo: `src/main.tsx`
- Ubicación: ANTES de `ReactDOM.render()` (fail-fast)
- Import: `import { env } from './config/env'`
- Efecto: Si validación falla → app NO arranca + error en consola

TESTING VALIDATION:

1. Test válido: `.env` con `VITE_MAX_CART_ITEMS=99` → app arranca OK
2. Test inválido (alto): `.env` con `VITE_MAX_CART_ITEMS=1000`
 - Error: `"Number must be less than or equal to 999"`

- App NO arranca
3. Test inválido (bajo): `.env` con `VITE_MAX_CART_ITEMS=0`
 - Error: "Number must be greater than or equal to 1"
 4. Test missing: Remover variable de `.env`
 - Error: "Expected string, received undefined"

VALIDACIÓN: App debe fallar en startup con mensaje claro si env var es invá

Aprende: Runtime validation con Zod previene

bugs de configuración en producción

Validación temprana detecta misconfiguraciones antes de runtime

Puntos Clave

1. **Nunca hardcodear:** Usar variables de entorno
2. **Validar temprano:** Schema Zod al inicio
3. **Diferente por entorno:** Secretos dev/staging/prod
4. **Frontend vs backend:** Prefijo `VITE_` para cliente
5. **Estrategia de rotación:** Actualizaciones sin downtime
6. **Nunca loggear:** No exponer secretos en logs/errors
7. **Auditar acceso:** Rastrear cuándo se usan secretos